Sifting Real Developer Conversations into Tasks: What Software Agents Miss in Practice

Haomin Qi, Xingliang Wang, Siyu Chen, Xin Zhang, Yu Kang, Pengfei Gao
Qingwei Lin, Saravan Rajmohan
Microsoft

***Abstract*—Developers ask software agents to change code, and they also ask them to diagnose incidents, explain architectures, review changes, and plan work. In these Analytical tasks, the agent delivers an answer, and the answer is useful only if evidence supports it. In industrial development, that evidence lives in internal services, runtime records, and access rules. Developers also rarely state a request in one message. They add details over several turns of a session as the work unfolds. Current benchmarks seldom capture either aspect. We present TaskSift, a framework that turns historical developer-agent sessions into executable evaluation cases. It recovers each request, records the turn at which each piece of user information became available, and grounds acceptance criteria in source evidence, both for Analytical tasks and for Modification tasks that change code. From 12,660 Microsoft internal product and open source sessions, TaskSift builds SiftBench, with 6,952 cases in ten categories. Two thirds of the internal product cases are Analytical. Evaluating four agent configurations yields three findings. First, Analytical tasks remain hard. Every configuration completes fewer Analytical than Modification tasks, and on internal product tasks the agents complete 56.0% of Analytical tasks against 62.8% of Modification tasks. Second, internal product Analytical tasks are completed about 17 percentage points less often than open source ones. Third, when the same task is given over several turns instead of in a single turn, completion drops by 11.9 points on internal product tasks and by 5.5 points on open source tasks.**

***Index Terms*—software agent, large language models, benchmarking, coding task construction**

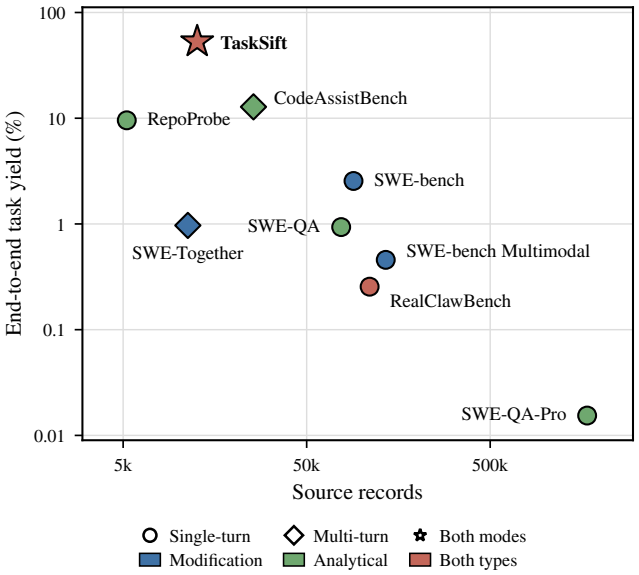


Fig. 1. End-to-end task yield of construction pipelines, in final tasks per source record, on logarithmic axes. Source records include sessions, issues, discussions, and pull requests. Yield accounts for filtering, sampling, and source-guided authoring. Marker shape encodes interaction mode and color encodes task type.

I. Introduction

Software agents now take part in much of daily development work. Developers ask them to write and fix code, and they also ask them to explain unfamiliar code, debug failures, review changes, and suggest solutions [1], [2]. We group these requests by what they ask the agent to produce. *Modification tasks* change code, configuration, or maintained documentation. *Analytical tasks* deliver explanations, diagnoses, reviews, plans, or query results. The two kinds of work are closely linked. A diagnosis often precedes a fix, and an architecture explanation often guides an implementation. Both can require repository exploration, command execution, and tool use.

The two kinds of tasks differ in how success is established. Tests can decide whether a patch is correct. An Analytical answer is acceptable only when evidence supports its conclusions and when it covers the conditions that the developer's next decision depends on. A diagnosis that names a plausible cause without the observation that distinguishes it from alternatives does not let the developer act.

Analytical work is especially common in industrial product development. Requests there refer to internal service dependencies, runtime records, ownership, and permissions that no public repository contains. Developers also rarely state every requirement in the first message. They refine requests and react to intermediate answers during a session [1]. Existing benchmarks cover parts of this work. SWE-bench and its successors evaluate code changes against tests [3], [4], and question answering, review, and diagnosis benchmarks each assess one kind of Analytical task [5]–[8]. Benchmarks built from real agent use, such as RealClawBench, SWE-Together, and CodeAssistBench, each cover one task type or one interaction mode [9]–[11]. None combines both task types, single-turn and multi-turn presentation, and industrial sources (Section II).

Historical sessions between developers and agents are a natural source for such a benchmark. A session records the initial request, later clarifications, observations, feedback, and decisions. Open source collections such as SWE-chat make these records available at scale [12]. We add sessions collected

from 23 Microsoft internal product repositories, which we call *internal product* sessions. Together, the two sources capture requests that were completed after one instruction and requests that developed through continued interaction.

Turning a session into a reliable evaluation case raises three challenges. The first is intent. A session may contain several goals, one goal may span several sessions, and later messages may revise earlier requirements. The construction must decide which messages belong to one request. The second is acceptance. A Modification task needs behavioral and regression checks. An Analytical task needs evidence for each fact, relationship, and conclusion the developer asked for. Code, tests, specifications, and runtime records must supply that evidence. The third is timing. Feedback and decisions that arrived late in the session must reach the agent only at the corresponding step. Otherwise, the initial request would contain later answers and corrective hints.

We address these challenges with TaskSift. TaskSift first identifies a developer request, collects the messages that define its requirements, and writes the task query. When all requirements are known at the start, the query is a single-turn request. Otherwise, it consists of an initial request and a user policy that releases later responses when the agent reaches the matching step. TaskSift then prepares the repository and supporting artifacts and derives acceptance criteria from the recorded requirements and source evidence. Modification tasks receive behavioral and regression checks. Analytical tasks receive checks for required facts, relationships, and supporting evidence. Finally, TaskSift validates the environment, the checks, and the user policy before it freezes the case.

Applied to 12,660 sessions from internal product repositories and from the open source collections DataClaw [13], Pi-staging [14], and SWE-chat [12], TaskSift produces SiftBench. The benchmark contains 6,952 admitted cases in ten task categories, of which 6,713 are execution-ready. TaskSift keeps about one execution-ready case for every two source sessions, a higher yield than any construction pipeline compared in Figure 1. Analytical tasks make up 67.0% of internal product cases and 28.0% of open source cases, so the industrial sessions widen the range of work the benchmark covers.

We evaluate four agent configurations on 1,560 randomly sampled cases and on a separate cohort of 480 paired task families. Three results stand out. First, Analytical tasks are harder than Modification tasks for every configuration. On internal product tasks, the agents complete 56.0% of Analytical tasks and 62.8% of Modification tasks. Second, the completion rate on internal product Analytical tasks is about 17 percentage points below the open source rate. Third, within paired families that share goals, evidence, and acceptance criteria, staged delivery of user information lowers completion by 11.9 percentage points on internal product tasks and by 5.5 points on open source tasks. A closer analysis of incomplete Analytical answers shows that they often contain individually supported statements but omit a dependency, an incident observation, or an authorization that the requested decision requires.

This paper makes three contributions.

- **Task construction from developer sessions.** TaskSift identifies developer requests in historical sessions, prepares their project context, preserves when user information becomes available, and derives evidence-grounded acceptance criteria for Modification and Analytical tasks.
- **An industrial and open source benchmark.** SiftBench combines internal product and open source sessions into ten categories of Modification and Analytical tasks, and its open source portion, including cases, environments, checkers, and user policies, will be made available.
- **An empirical study of what agents miss.** We evaluate four agent configurations and show where Analytical tasks remain incomplete, how internal product tasks differ from open source ones, and how staged user interaction affects completion.

II. Motivation

Incident diagnosis is the largest category among internal product cases (20.3%, Section IV). Consider a typical request of this kind, which we also use as Example 2 (Figure 3b). A developer asks the agent why one email was not processed. The repository shows how messages are normally processed, stage by stage. An answer that walks through this path, cites the relevant code, and summarizes an aggregate queue snapshot reads well and may contain only true statements. It still does not tell the developer what happened to this message. To answer that, the agent must ask for the message identifier, find the log entries and runtime records for that message, and establish which processing stages it actually reached. The queue snapshot supports only a statement about the queue. When the evidence stops short of a cause, a useful answer says so and names the observation that would separate the remaining explanations. Access also matters. The agent may request a production secret, which the developer would not provide.

This request shows three properties that are common in industrial work and rare in existing benchmarks. First, the expected output is an analysis, and its correctness depends on evidence and on the conditions the developer needs for the next step. No test suite decides it. Second, the evidence lives in internal services, logs, and access rules that no public repository contains. Third, the information arrives during the session. The developer supplies the message identifier only after the agent asks for it, so an evaluation that puts every detail into the first prompt removes part of the task.

Table I compares existing benchmarks along these properties. Issue resolution benchmarks evaluate Modification tasks with single-turn requests [3], [4], [15], [16]. Understanding, diagnosis, and review benchmarks each target one kind of Analytical task [5]–[8], [18]. Benchmarks built from real interactions come closest [9]–[11], yet each covers one task type or one interaction mode. Only SWE-Bench Pro and RealClawBench include industrial sources, and both present requests in a single turn [4], [9]. No existing benchmark combines Analytical and Modification tasks, single-turn and

TABLE I

TASK SCOPE OF RELATED BENCHMARKS. SINGLE-TURN CASES PROVIDE THE FULL REQUEST UPFRONT, AND MULTI-TURN CASES ADD LATER USER INPUT. MODIFICATION TASKS CHANGE REPOSITORY ARTIFACTS, AND ANALYTICAL TASKS DELIVER EXPLANATIONS OR DECISIONS. INDUSTRIAL MARKS BENCHMARKS WITH TASKS FROM PROPRIETARY OR COMPANY-INTERNAL SOURCES. ORIGIN SEPARATES REAL DEVELOPER REQUESTS FROM SYNTHETIC AUTHORING.

Work	Single-turn	Multi-turn	Modification tasks	Analytical tasks	Industrial	Origin
Issue resolution and task synthesis						
SWE-bench [3]	✓	✗	✓	✗	✗	Real-world
SWE-Bench Pro [4]	✓	✗	✓	✗	✓	Real-world
SWE-bench Multimodal [15]	✓	✗	✓	✗	✗	Real-world
SWE-smith [16]	✓	✗	✓	✗	✗	Synthetic
SWE-Lancer [17]	✓	✗	✓	✓	✗	Real-world
Interaction-derived task construction						
SWE-Together [10]	✗	✓	✓	✗	✗	Real-world
RealClawBench [9]	✓	✗	✓	✓	✓	Real-world
CodeAssistBench [11]	✗	✓	✗	✓	✗	Real-world
Understanding, diagnosis, and review						
SWE-QA [5]	✓	✗	✗	✓	✗	Synthetic
SWE-QA-Pro [18]	✓	✗	✗	✓	✗	Synthetic
RepoProbe [6]	✓	✗	✗	✓	✗	Real-world
ORCA-bench [8]	✓	✗	✗	✓	✗	Synthetic
ReviewBench [7]	✓	✗	✗	✓	✗	Real-world
TaskSift (ours)	✓	✓	✓	✓	✓	**Real-world**

multi-turn presentation, and industrial sources. TaskSift is designed to fill this gap.

III. TASKSIFT DESIGN

TaskSift turns a historical session into an evaluation case in three stages (Figure 2). It recovers the developer's intent and writes the task query, grounds acceptance criteria in source evidence and builds an executable case, and finally runs an agent on the frozen case and checks its output against those criteria.

Every case has a task type and an interaction mode. *Modification tasks* produce patches or replacement files. *Analytical tasks* produce explanations, diagnoses, reviews, plans, or query results. A single-turn case states the full request upfront, and a multi-turn case releases later observations, decisions, or authorizations during the attempt. We represent a case as

$$\mathcal{C} = (q,\, X,\, \pi,\, \mathcal{A},\, V), \tag{1}$$

where q is the initial query, X the versioned inputs such as the repository and runtime records, π the user policy that decides which information to release and when, $\mathcal{A}$ the set of acceptance claims, and V the checking procedure of each claim. The task type determines $\mathcal{A}$ and V, and the interaction mode determines q and π. A single-turn case has an empty policy.

A. Recovering Intent and Writing Queries

Construction keeps every message, tool call, observation, and attachment in event order, with content hashes that reveal duplicates and continued conversations. Developer requests and accepted decisions define the intent, and historical agent messages only point to candidate evidence. A task corresponds to one goal that can be delivered and assessed on its own. A diagnosis and a repair requested separately become two linked tasks, and related variants share a task family identifier.

For each task, a *requirement ledger* records goals, constraints, observations, preferences, authorizations, and expected outputs, each with its source message, the turn at which it first became available, and whether it is still active. The ledger is the single record from which both the query and the acceptance claims are derived.

Single-turn queries. A single-turn query combines the requirements available before execution began. A requirement enters only if it does not depend on intermediate results and does not reveal the solution, so corrections that restate a known constraint are kept and corrections that reveal the fix stay private. In Example 1 (Figure 3a), the marker position, the rejection of reply and forward prefixes, and the consistency across entry points are all known upfront and form one request.

Multi-turn queries. A multi-turn query pairs the first coherent request with a user policy π that holds the evidence the developer supplied on request, the decisions that depended on investigation, and the authorizations granted during the session. In Example 2 (Figure 3b), the agent must ask for the message identifier and the available observations, while inputs it can inspect with its own tools stay directly accessible. When the requirements of a family can be stated upfront without revealing the solution, TaskSift builds both views from the same ledger. A single-turn view of Example 2 would state the message identifier, the log observations, and the queue snapshot in the initial query. Its multi-turn view gives only the request and releases each item when the agent asks for it, in

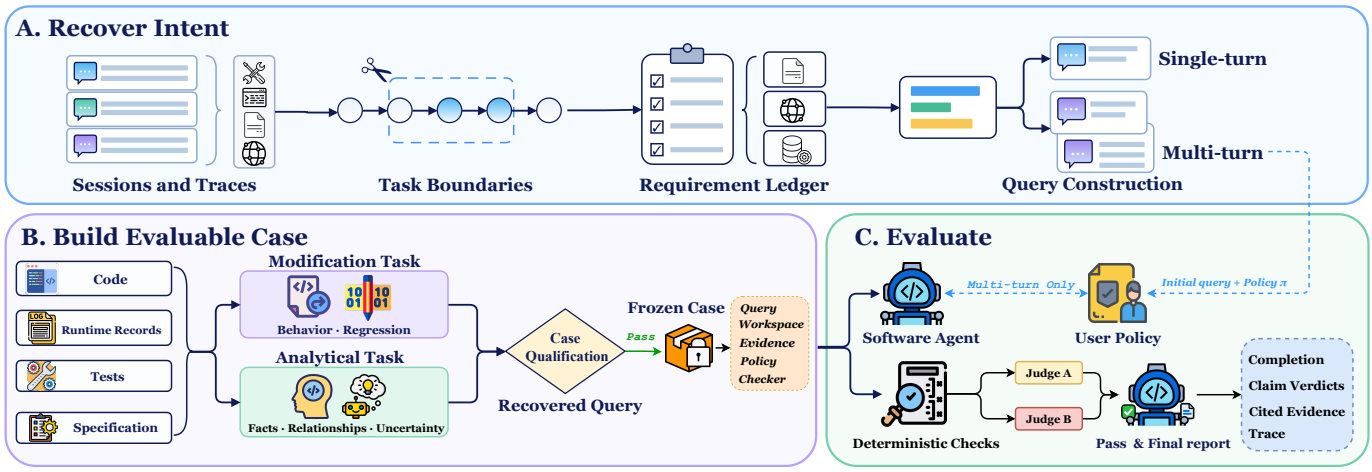


Fig. 2. TaskSift pipeline. (A) Intent recovery turns sessions into a requirement ledger and single-turn or multi-turn queries. (B) Case construction grounds acceptance checks in evidence according to task type and qualifies the frozen case. (C) Evaluation runs the agent with the user policy, applies deterministic checks and two model-based judges, and reports completion with claim verdicts.

the order the developer supplied it. Both views share the same acceptance claims, so a difference in completion reflects only the timing of information. Query review checks requirement coverage, input timing, family identity, and answer leakage.

B. Grounding Acceptance Criteria

Every case fixes its repository revision and, where relevant, its data snapshot and incident window. Acceptance evidence comes from code, specifications, tests, historical changes, runtime records, and constraints the developer confirmed. Each mandatory requirement becomes an atomic claim $a_i \in \mathcal{A}$ in a *Ground Truth Card*, which links the claim to its originating requirement r_i, its supporting evidence e_i, and its checking procedure $v_i \in V$, and records acceptable alternatives and prohibited overclaims. Construction models propose claims and locate evidence, and independent sources must confirm each claim, so every verdict can be traced back to a requirement.

Modification tasks. Tests and builds check the target behavior and regressions, and static checks and grounded review check interfaces, scope, and authorization. Historical patches identify the intended behavior, and any equivalent implementation passes the same checks. Following EvalPlus [19], we add tests that separate correct implementations from plausible incorrect ones. In Example 1, the checks tagged R1 to R3 accept the marker at either boundary, reject reply and forward prefixes, and apply both rules at the trigger and the receiver.

Analytical tasks. Checks follow the decision the developer requested. Understanding tasks require the relevant facts and relationships, reviews require supported findings, and plans require feasible actions, permissions, and dependencies. Query tasks use a deterministic oracle for the exact result set. A diagnosis must cite observations that distinguish candidate causes. In Example 2, claims C1 to C3 require the stages this message reached, support and scope for every statement, and the next discriminating evidence when the cause remains open. An aggregate queue snapshot supports only a statement about the queue. Multi-turn cases add the claims of each decision the policy permits and keep all earlier constraints.

C. Building Executable Cases

A case bundle contains the query, the workspace, the submission format, the user policy, the private acceptance evidence, the checker, and a hash manifest. Artifacts are initially visible, released on request, or evaluator-only, and historical solutions and later feedback are evaluator-only. Environments are rebuilt by replaying historical revisions, by structural replay with anonymized records and mock services, or by model-assisted repair of paths and interfaces that have drifted. We build and test repositories with RepoLaunch, an agentic framework that resolves dependencies, compiles code, and extracts test results across languages and platforms [20]. Model-assisted repair shares its goal with Change2Task, which reconstructs task states on healthy recent revisions of the same repository [21]. Fidelity records mark which material is historical and which is reproduced.

User policies. Each policy transition specifies an observable trigger, the response or decision it releases, its effect on authorization, and a stopping rule. One request can fire several transitions, and alternative valid paths have transitions of their own. The policy selects or paraphrases approved responses from frozen material, and a correction fires only when the corresponding error occurs in the current attempt. In Example 2, the policy releases the message identifier, the log observations, and the queue snapshot only in response to matching requests, and it declines a request for a production secret.

Case qualification. Before admission, each checker is exercised on correct, incorrect, and equivalent-correct submissions. Modification controls add faulty behavior and regressions. Analytical controls add paraphrases, omitted facts, unsupported causes, and justified uncertainty. Policy controls test premature disclosure, omitted responses, and alternative paths.

(a) Example 1 | Single-turn Modification task

Request: accept tagged subjects, but not replies or forwards.

Recorded requirements

R1 The marker counts only at the start or end of the subject.

R2 Reply and forward prefixes are rejected, in any letter case.

R3 The trigger and the receiver apply the same rules.

```
# Checks derived from R1 to R3 (illustrative)
examples = [
  ("[fishnotes] topic", True),                    # R1 start
  ("topic [fishnotes]", True),                      # R1 end
  ("x [fishnotes] y", False),                    # R1 middle
  ("RE: [fishnotes] x", False),                   # R2 reply
  ("FW: [FISHNotes] x", False),                  # R2 forward
]
for gate in (trigger_gate, receiver_gate):                # R3
    for subject, expected in examples:
        assert gate(subject) == expected
```

PASS Both entry points return the expected result for every example.

(b) Example 2 | Multi-turn Analytical task

Request: explain why one message was not processed.

Recorded requirements

P Message ID, logs, and queue snapshot are released only on request.

C1 The answer states which stages this message reached.

C2 Every claim is supported and stays within its evidence scope.

C3 Without a proven cause, it names the next discriminating evidence.

```
# User policy during the attempt (P)
on request(item in {message_id, logs, queue_snapshot}):
    release(item)
on request(production_secret):
    decline()
# Checks on the final answer (illustrative)
check_stages_reached(answer, message_logs)                    # C1
for claim in answer.claims:
    check_support_and_scope(claim, released)                  # C2
if not answer.cause_established:
    require(answer.open_alternatives, answer.next_evidence)   # C3
```

PASS Stages covered, claims supported within scope, uncertainty bounded.

Fig. 3. Acceptance checks for (a) a single-turn Modification task and (b) a multi-turn Analytical task. Each panel lists the requirements recorded from the session. The illustrative pseudocode tags every check with the requirement it enforces, and panel (b) also shows the user policy that releases evidence during the attempt.

Privacy checks, a visibility review, and clean resets complete qualification, and any change creates a new case version.

D. Executing and Evaluating Cases

Each attempt starts from a fresh workspace with a fixed baseline and allowlisted access. The agent returns a patch, replacement files, or an answer with the required fields, and the host applies changes to an isolated copy for testing. A multi-turn attempt keeps its conversation state while the policy releases eligible artifacts, and every clarification, decision, and authorization is logged.

Evaluation first runs the deterministic checks, and a failed mandatory check means the attempt is not completed. Semantic claims go to two independently invoked large language model (LLM) judges that receive the same frozen criterion packet. A coverage judge assesses evidence sufficiency and coverage, and a contradiction judge searches for contradictions and unsupported statements. Each judge sees the candidate output, one atomic claim, its authorized evidence, and the acceptable alternatives, but not the other judge's verdict, and must cite evidence for a decisive verdict. A claim passes when both judges find it supported. Requiring both verdicts makes a pass depend on support and on the absence of error, and judges from different model families are less likely to share a blind spot.

Let τ be an attempt and $\mathcal{M}(\tau) \subseteq \mathcal{A}$ the mandatory claims that apply to it, which for a multi-turn attempt include the claims of approved decisions and required stages. With $v_i(\tau, e_i)$ the verdict for claim a_i given its evidence e_i, an attempt is completed if and only if

$$\text{Solved}(\tau) \iff \forall i \in \mathcal{M}(\tau), \quad v_i(\tau, e_i) = \text{PASS}. \tag{2}$$

Each result keeps the case version, the released artifacts, the submission, the checker outputs, the verdicts, and the full trace, following the documentation practices of Datasheets for Datasets and the Holistic Evaluation of Language Models (HELM) [22], [23].

IV. Experiment and Analysis

A. Research Questions and Setup

We organize the evaluation around four research questions (RQs). Each RQ uses its own set of cases and its own counting unit, and we introduce them in that order.

RQ1 concerns construction. It asks what developer work TaskSift preserves and how reliable the resulting cases are, so its unit is the source session. The construction corpus contains 2,184 internal product sessions from 23 repositories and 10,476 open source sessions from DataClaw (2,228), Pi-staging (2,397), and SWE-chat (5,851). A session yields one or more *cases*, each built for one developer request. A case is *admitted* when it passes qualification (Section III) and *execution-ready* when its environment also runs end to end. The agent experiments below use execution-ready cases only. We report construction yield, the number of cases per source session, and session coverage, the share of sessions that contribute at least one case. To measure case quality, we also audit the same set of construction proposals under TaskSift and two simpler constructors, as described in RQ1.

RQ2 asks how well agents complete the Modification and Analytical tasks that developers actually requested. Before any agent ran, we drew a *natural cohort* by simple random sampling without replacement from the execution-ready cases: 360 internal product cases and 1,200 open source cases, 1,560 in total. Random sampling keeps the task mix of each source, including its share of Analytical tasks and of multi-turn cases. We evaluate four agent configurations, each pairing a command-line interface (CLI) harness with a model: Codex

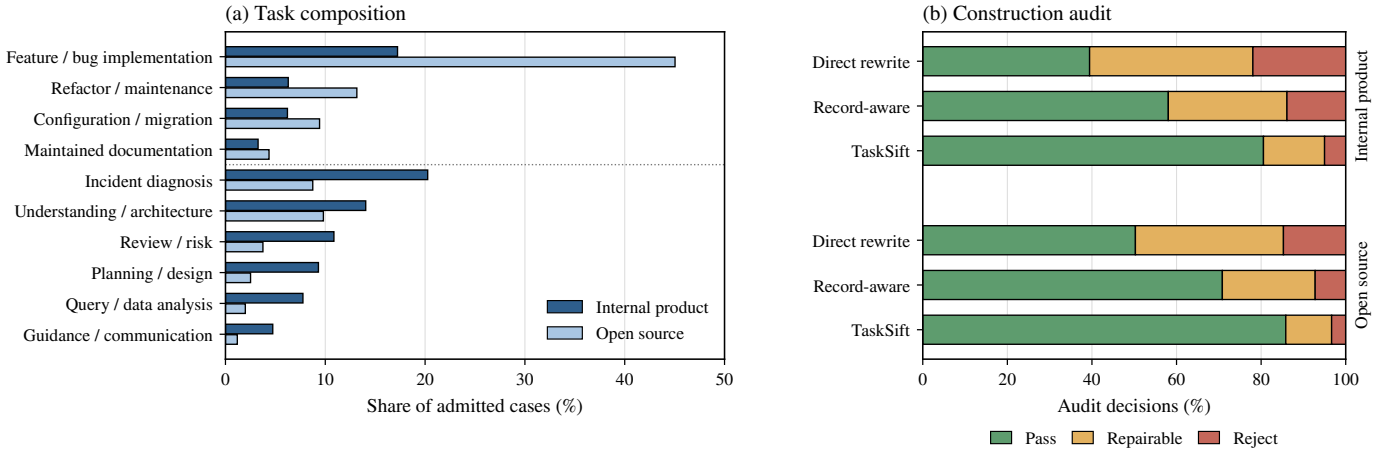


Fig. 4. (a) Composition of admitted cases by category. The dotted line separates Modification from Analytical categories. (b) Construction audit on the same 360 proposals per source.

TABLE II
SIFTBENCH INVENTORY. (A) SOURCE SESSIONS AND CONSTRUCTED CASES. (B) ADMITTED CASES BY CATEGORY. MOD. AND ANAL. DENOTE MODIFICATION AND ANALYTICAL TASKS.

(a) Sessions and constructed cases

Source	Sessions	Cases	Mod.	Anal.
Internal product	2,184	1,224	404	820
DataClaw	2,228	1,196	837	359
Pi-staging	2,397	1,348	984	364
SWE-chat	5,851	3,184	2,303	881
Open source subtotal	10,476	5,728	4,124	1,604
All sources	**12,660**	**6,952**	**4,528**	**2,424**

Construction summary	Internal product	Open source	All
Execution-ready cases	1,187	5,526	6,713
Supporting sessions	1,160	5,379	6,539
Task categories	10	10	10

(b) Task categories

Category	Internal product	Open source	All
Modification tasks (4 categories)			
Feature / bug implementation	211	2,580	2,791
Refactor / maintenance	77	754	831
Configuration / migration	76	540	616
Maintained documentation	40	250	290
Analytical tasks (6 categories)			
Incident diagnosis	248	501	749
Understanding / architecture	172	562	734
Review / risk	133	215	348
Planning / design	114	144	258
Query / data analysis	95	114	209
Guidance / communication	58	68	126
All categories	**1,224**	**5,728**	**6,952**

CLI with GPT-6.1-sol, Codex CLI with GPT-5.6-luna, Copilot CLI with GPT-6-astra, and Grok CLI with Grok 4.7. All configurations receive the same inputs, permissions, acceptance criteria, and a 128k-token budget. Each configuration runs inside the case workspace and receives the case query as its first message. In a multi-turn case, the user policy answers the agent's messages and releases information when the agent asks for it. The run ends when the agent submits its patch or answer, or when the budget is exhausted. A *run* is one attempt by one configuration on one case. Each configuration attempts every case once, which gives 1,440 internal product runs and 4,800 open source runs. A run is *completed* when every applicable mandatory claim passes. Completion rates divide completed runs by attempted runs. Aggregate rates pool the four configurations, so every run has equal weight.

RQ3 asks how staged user interaction affects completion. The natural cohort alone cannot answer this question. Its single-turn and multi-turn cases are different tasks, so a difference in completion could come from task content as well as from interaction. For this reason, we build a separate *paired cohort* of task families. A *task family* groups the cases derived from one developer request. For a family in this cohort, TaskSift builds two *views*. The single-turn view states every requirement in the initial query. The multi-turn view states the initial request and releases the remaining requirements through the user policy, at the points where they arrived in the original session. Both views share the same goals, evidence, and acceptance criteria, so they differ only in when information reaches the agent. We include only families whose requirements can be stated upfront without revealing the solution. The paired cohort contains 180 internal product and 300 open source families, 480 in total, and does not overlap with the natural cohort. A *pair* is one family attempted by one configuration in both views. Four configurations give 720

TABLE III

COMPLETION (%) AT A 128K-TOKEN BUDGET. N IS THE NUMBER OF CASES PER CONFIGURATION. MOD. AND ANAL. DENOTE MODIFICATION AND ANALYTICAL TASKS. AGGREGATE ROWS POOL THE FOUR CONFIGURATIONS. SOURCE GAP IS INTERNAL PRODUCT MINUS OPEN SOURCE ANALYTICAL COMPLETION, IN PERCENTAGE POINTS.

	Internal product			Open source			Source gap
Agent / model	Mod.	Anal.	All	Mod.	Anal.	All	Anal. (pp)
			Single-turn				
N (cases)	83	168	251	650	250	900	
Codex CLI / GPT-5.6-luna	59.0	51.8	54.2	69.5	66.0	68.6	-14.2
Codex CLI / GPT-6.1-sol	68.7	63.7	65.3	79.1	78.0	78.8	-14.3
Copilot CLI / GPT-6-astra	74.7	69.6	71.3	84.3	84.4	84.3	-14.8
Grok CLI / Grok 4.7	63.9	57.7	59.8	74.9	72.0	74.1	-14.3
Aggregate	**66.6**	**60.7**	**62.6**	**77.0**	**75.1**	**76.4**	**-14.4**
			Multi-turn				
N (cases)	36	73	109	214	86	300	
Codex CLI / GPT-5.6-luna	41.7	32.9	35.8	64.0	54.7	61.3	-21.8
Codex CLI / GPT-6.1-sol	58.3	49.3	52.3	74.8	70.9	73.7	-21.6
Copilot CLI / GPT-6-astra	66.7	57.5	60.6	80.8	77.9	80.0	-20.4
Grok CLI / Grok 4.7	50.0	41.1	44.0	69.2	62.8	67.3	-21.7
Aggregate	**54.2**	**45.2**	**48.2**	**72.2**	**66.6**	**70.6**	**-21.4**
			All natural tasks				
N (cases)	119	241	360	864	336	1,200	
Codex CLI / GPT-5.6-luna	53.8	46.1	48.6	68.2	63.1	66.8	-17.0
Codex CLI / GPT-6.1-sol	65.5	59.3	61.4	78.0	76.2	77.5	-16.9
Copilot CLI / GPT-6-astra	72.3	66.0	68.1	83.4	82.7	83.2	-16.8
Grok CLI / Grok 4.7	59.7	52.7	55.0	73.5	69.6	72.4	-16.9
Aggregate	**62.8**	**56.0**	**58.3**	**75.8**	**72.9**	**75.0**	**-16.9**

internal product pairs and 1,200 open source pairs, or 3,840 runs. For each pair, we record whether the agent completes both views, only one of them, or neither.

RQ4 asks what incomplete Analytical answers miss. It examines the Analytical runs of the natural cohort that are not completed. For each such run, the Ground Truth Card of its case shows which mandatory claims failed. This links the completion rates of RQ2 and RQ3 to the specific evidence and conditions that the answers omitted.

All RQs apply the acceptance procedure of Section III. Deterministic checks run first, and semantic claims go to the two-judge protocol, with GPT-5.4 as the coverage judge and Gemini-3.8 Flash as the contradiction judge. To validate the judges, two annotators independently labeled 430 claims from 200 Analytical runs, stratified by source, category, interaction mode, and configuration, and resolved disagreements through adjudication. The annotators agree with each other at Cohen's $\kappa = 0.80$ on claims and 0.86 on runs. Against the adjudicated labels, the final judge verdicts agree on 90.7% of claims over the three labels supported, contradicted, and undecidable ($\kappa = 0.78$) and 89.0% of runs ($\kappa = 0.78$). Only 6 of the 309 claims the judges pass (1.9%) lack human support. The judges also lean conservative. They mark 16 runs incomplete that the annotators accept and 6 runs complete that the annotators reject, so the reported completion rates, if anything, understate agent performance. Human labels preserve the ranking of the four configurations.

B. RQ1: What Work Is Preserved, and How Reliably?

Table II traces the four source collections to the admitted cases. TaskSift admits 1,224 internal product and 5,728 open source cases, of which 1,187 and 5,526 are execution-ready. Nearly all admitted cases run end to end, 97.0% of internal product and 96.5% of open source cases, because qualification already exercises each environment and checker. This amounts to 56.0% and 54.7% admitted cases per session, and 53.1% and 51.3% of sessions contribute at least one case. With 6,713 execution-ready cases from 12,660 sessions, TaskSift has the highest end-to-end yield in Figure 1, and it is the only pipeline there that covers both task types and both interaction modes.

Table IIb groups the cases into ten categories. The four Modification categories add or fix behavior, restructure code without changing behavior, change configuration or migrate between versions and systems, and update maintained documentation. The six Analytical categories explain a failure from runtime evidence, explain how code works and how components relate, assess a change or design for defects and risks, propose a plan or design under constraints, compute an answer from data or logs, and advise developers or draft explanations for others. The industrial sessions change what the benchmark measures. Analytical tasks make up 820 of 1,224 internal product cases (67.0%) and 1,604 of 5,728 open source cases (28.0%). Incident diagnosis alone accounts for 20.3% of internal product cases, while feature and bug implementation accounts for 45.0% of open source cases (Figure 4a). Internal product cases also spread more evenly

over the ten categories. Their effective category count, the exponentiated Shannon entropy of the category distribution, is 8.72, compared with 5.89 for open source cases.

The construction audit compares three constructors on the same 360 proposals per source, sampled before qualification (Figure 4b). Direct rewriting turns a conversation into a task. Record-aware construction adds repository context. TaskSift adds the requirement ledger, grounded acceptance criteria, and qualification. The audit checks five properties of each proposal. Intent asks whether the query matches what the developer wanted. Acceptance criteria asks whether every claim is grounded in evidence. Inputs asks whether the case provides the artifacts the task needs. Interaction asks whether information is released at the right point. Leakage asks whether the query or the visible artifacts reveal the solution. GPT-5.4 and Gemini-3.8 Flash independently judge each proposal for intent, acceptance criteria, inputs, interaction, and answer leakage without knowing which constructor produced it, and a frozen rule assigns pass, repairable, or reject. TaskSift passes 80.6% of internal product proposals, compared with 58.1% for record-aware construction and 39.4% for direct rewriting. The open source pass rates are 85.8%, 70.8%, and 50.3%. A human audit of 160 of these proposals confirms the ordering. Two annotators agree at quadratic weighted $\kappa = 0.81$, the judges match their adjudicated labels on 82.5% of proposals (weighted $\kappa = 0.73$), and the human pass rates are 80.8%, 59.3%, and 37.0% on internal product proposals and 84.6%, 70.4%, and 51.9% on open source proposals.

Finding 1. TaskSift yields about one execution-ready case for every two sessions. Industrial sessions shift the benchmark toward Analytical work, which makes up two thirds of internal product cases. Grounded requirements and qualification double the audit pass rate of direct rewriting on internal product sessions.

C. RQ2: How Do Agents Perform on Modification and Analytical Tasks?

Table III reports completion by configuration, source, and interaction mode. On internal product tasks, the agents complete 299 of 476 Modification runs (62.8%) and 540 of 964 Analytical runs (56.0%). On open source tasks, they complete 2,619 of 3,456 Modification runs (75.8%) and 980 of 1,344 Analytical runs (72.9%). Analytical tasks trail Modification tasks by 6.8 percentage points (pp) on internal product tasks and by 2.9 pp on open source tasks, and all four configurations show this ordering in both sources. Figure 5 shows that internal product tasks are harder in every combination of task type and interaction mode.

The configurations also differ in completion. On internal product Analytical tasks, completion ranges from 46.1% for Codex CLI with GPT-5.6-luna to 66.0% for Copilot CLI with GPT-6-astra, with GPT-6.1-sol at 59.3% and Grok 4.7 at 52.7%. On open source Analytical tasks, the range is 63.1% to 82.7%, with the same ranking. Pooled over the configurations, internal product Analytical tasks are completed at 56.0%, 16.9 pp below the 72.9% of open source Analytical tasks.

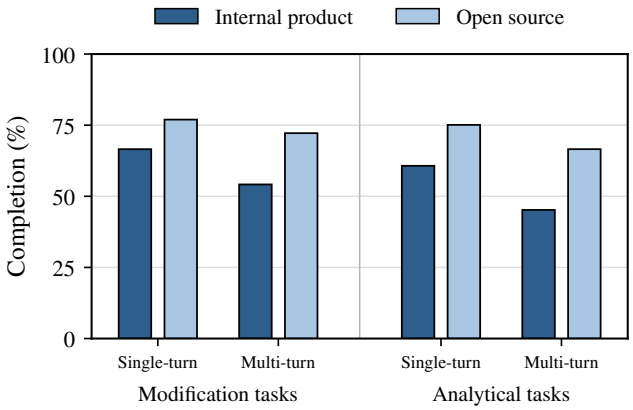


Fig. 5. Completion by task type, interaction mode, and source, pooled over the four agent configurations.

Finding 2. Analytical tasks are harder than Modification tasks for every configuration. Internal product Analytical tasks are completed about 17 pp less often than open source ones.

D. RQ3: How Does Interaction Affect Completion?

In the natural cohort, internal product completion is 62.6% for single-turn and 48.2% for multi-turn cases, and open source completion is 76.4% and 70.6%. These cases differ in content as well as presentation. Internal product multi-turn cases have a median of eight mandatory requirements, compared with five for single-turn cases. In 56.0% of them, the answer must link a specific event, such as one unprocessed message, to the code path, log entries, and runtime records that show what happened to it, compared with 29.1% of single-turn cases.

The paired cohort removes this difference. On internal product tasks, completion falls from 455 of 720 pairs (63.2%) in the single-turn view to 369 of 720 (51.3%) in the multi-turn view. On open source tasks, it falls from 924 of 1,200 (77.0%) to 858 of 1,200 (71.5%). The paired gaps are 11.9 and 5.5 pp (Figure 6). Of the 720 internal product pairs, 335 succeed in both views, 120 only in the single-turn view, 34 only in the multi-turn view, and 231 in neither. The open source counts are 817, 107, 41, and 235. Among the pairs that succeed in at least one view, 24.5% (120 of 489) succeed only when all information is given upfront on internal product tasks, compared with 11.1% (107 of 965) on open source tasks. In the multi-turn view, the agent must keep earlier constraints, integrate evidence released later, and follow decisions that arrive during the attempt.

The effect holds for every configuration. Internal product losses range from 6.1 pp for Copilot CLI with GPT-6-astra to 15.0 pp for Codex CLI with GPT-5.6-luna, with GPT-6.1-sol at 12.2 pp and Grok 4.7 at 14.4 pp. Open source losses range from 3.3 to 7.0 pp, and each configuration loses more on internal product tasks.

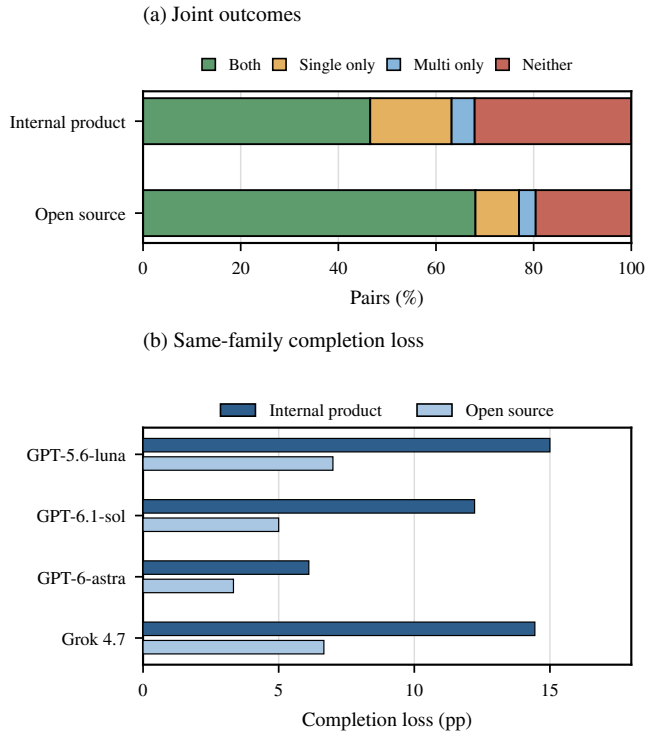


Fig. 6. Same-family comparisons. (a) Joint completion outcomes pooled over the four configurations. (b) Single-turn minus multi-turn completion for each configuration, in percentage points.

TABLE IV
INCOMPLETE ANALYTICAL RUNS OVER ATTEMPTED RUNS, POOLED OVER THE FOUR CONFIGURATIONS, BY SOURCE AND INTERACTION MODE.

	Internal product		Open source	
Mode	Count	%	Count	%
Single-turn	264/672	39.3	249/1,000	24.9
Multi-turn	160/292	54.8	115/344	33.4
All	424/964	44.0	364/1,344	27.1

Finding 3. With goals, evidence, and acceptance criteria held fixed, staged information delivery lowers completion for every configuration. The loss is about twice as large on internal product tasks as on open source tasks.

E. RQ4: What Do Incomplete Analytical Answers Miss?

Across the four configurations, 424 of 964 internal product Analytical runs and 364 of 1,344 open source Analytical runs are incomplete (Table IV). On internal product tasks, the incomplete share rises from 39.3% in single-turn to 54.8% in multi-turn execution. On open source tasks, it rises from 24.9% to 33.4%. The failed claims in their Ground Truth Cards fall into three kinds.

Decision-critical relationships. An architecture explanation must connect local components to the shared clients, dependencies, and resource boundaries that decide the question. Listing the components and citing their definitions answers only part of the request, and the claims that check coupling or isolation then fail.

Incident evidence. In Example 2, a repository path describes how a message could be processed, while message identifiers and correlated observations show which stages this message reached. A diagnosis must keep these scopes apart and name the next discriminating evidence when alternatives remain open.

Authorization. A plan that changes production data can require a mutation-free dry run, an impact estimate, and rollback evidence before approval. A service-isolation recommendation can require a check of shared processes and resources. An answer that recommends the change without these steps fails the corresponding claims, even when its technical analysis is correct. Internal product sessions supply these organizational and runtime conditions.

The 540 internal product and 980 open source Analytical runs that complete show that these criteria are attainable. A complete answer supports its claims, covers the relationships that matter, states uncertainty where the evidence calls for it, and names an authorized next action.

Finding 4. Supported facts alone do not make an actionable conclusion. Incomplete Analytical answers often miss the dependency, the incident evidence, or the authorization that the developer's decision requires.

V. LESSONS LEARNED

Evaluate agents on the work developers actually request. Two thirds of the internal product cases are Analytical tasks, and incident diagnosis is the largest single category. A team that judges agents only by patch tests leaves most of this work unmeasured. Open source results are still useful for choosing among agents, since all four configurations keep the same ranking on internal product and open source tasks. They are not a reliable estimate of internal completion, which is about 17 pp lower on Analytical tasks. Teams that adopt agents need to build evaluation cases from their own repositories, services, and access rules, accept Analytical answers only when evidence supports each required claim, and keep the timing of user information, since staged delivery alone costs 11.9 pp on internal product tasks.

Developer sessions are a practical source, but turning them into cases takes engineering. Sessions already contain the requests, clarifications, and evidence that an evaluation case needs, and TaskSift turns about one in two sessions into an execution-ready case. Three practical difficulties shaped how we got there. First, rebuilding environments was often harder than writing the task itself. Many repositories depend on complex build setups, internal services, and specific data states, and bringing one up for a single case could take longer than recovering the request and its acceptance criteria. We used RepoLaunch [20] to automate this step, since it resolves dependencies, builds the repository, and runs its tests. Structural replay with mock services covers internal services

that cannot be started outside production, and fidelity records mark what was reproduced. Second, internal sessions are noisy. They contain discussions unrelated to the task and long stretches in which the model explains its reasoning to the developer. Such content blurs where one request ends and the next begins and hides the signals that mark a new stage in a multi-turn task. The requirement ledger addresses this by recording only developer requirements and decisions, each with the turn at which it first appeared. Third, collecting developer trajectories inside a company raises privacy and compliance questions. We collected only content related to the target repository, with developer consent and under internal data-handling policies, which kept the internal cases usable for evaluation and allows the open source portion of SiftBench to be shared while internal results are reported in aggregate. Volume alone is not enough either. Directly rewriting conversations passes the construction audit for only 39.4% of internal product proposals, while grounding requirements and qualifying checkers and user policies raises the pass rate to 80.6%.

VI. Related Work

Benchmarks for software agents. Code generation benchmarks pair specifications with tests, from HumanEval and Mostly Basic Python Problems (MBPP) to LiveCodeBench, BigCodeBench, and CodeXGLUE [24]–[28]. RepoBench and CrossCodeEval add repository context [29], [30]. SWE-bench pairs GitHub issues with tests at repository scale [3], and SWE-Bench Pro, SWE-bench Multimodal, SWE-smith, and SWE-Lancer extend this setting to long-horizon, visual, synthesized, and freelance tasks [4], [15]–[17]. Change2Task converts merged pull requests into verified tasks on maintained revisions [21]. WitnessGym constructs bug-validation tasks by injecting bugs into real projects and retaining cases with executable witnesses [31]. SQBench scores task outputs in production-oriented workflows [32]. For Analytical tasks, InfiBench, SWE-QA, SWE-QA-Pro, and RepoProbe evaluate code question answering [5], [6], [18], [33], ReviewBench evaluates code review [7], and ORCA-bench evaluates oncall root cause analysis [8]. TaskSift brings these task types into one construction process and adds industrial sources and staged interaction (Table I).

Tasks from real usage. SWE-chat collects coding agent sessions from open source developers [12]. SWE-Together replays such sessions with a reactive simulated user [10], RealClawBench reconstructs environments from an internal OpenClaw deployment [9], and CodeAssistBench builds multi-turn assistance tasks from GitHub issues [11]. In industry, CursorBench draws tasks from production editor sessions [34], Tencent WorkBuddy Bench derives tasks from real commits and business scenarios [35], and BTS-AgentBench converts operational telemetry into tool-use tasks [36]. TaskSift differs in two ways. It derives evidence-grounded acceptance criteria for Analytical tasks as well as Modification tasks, and it records when each piece of user information became available so the same task can be presented in one turn or several.

Interaction and judging. Studies of multi-turn use show that models lose track of incrementally specified requests [37]. The τ-bench family and ToolSandbox couple user dialogue with tools and policies [38]–[40], and feedback-driven methods such as Reflexion and Self-Refine revise earlier attempts [41]–[43]. Agent frameworks such as ReAct, SWE-agent, OpenHands, and Agentless, and interactive benchmarks such as AgentBench, AgentBoard, WebArena, WorkArena, and OSWorld, model the actions an agent takes [44]–[52]. TaskSift also models the user turns that add requirements and decisions. For judging, G-Eval and Prometheus apply explicit criteria [53], [54], MT-Bench, JudgeBench, and LLMBar study judge reliability [55]–[57], and FActScore, ALCE, and SelfCheckGPT check factual support [58]–[60]. Gorinova et al. argue that agent benchmarks should separate the model from its harness and accept valid alternatives [61]. TaskSift reports each configuration separately, records acceptable alternatives in every claim, and checks that supported statements also cover the relationships a decision depends on.

VII. Threats to Validity

Semantic claims are judged by LLMs. To limit this threat, we decompose acceptance into atomic claims, run deterministic checks first, use two judges from different model families, and accept a semantic verdict only when both judges agree and cite evidence. A human validation on 430 claims shows 90.7% claim-level agreement, a false-pass rate of 1.9%, and a conservative bias in run completion. Each configuration makes one attempt per case, and pooled counts with consistent directions across configurations limit run-to-run variance. Multi-turn responses are drawn from the developers' own recorded messages, which keeps their content and order close to the original session. Internal product sessions come from one company. The construction process does not depend on Microsoft-specific infrastructure and applies to session records from other organizations. Finally, public repositories may appear in model training data, which could raise open source completion.

VIII. Conclusion and Outlook

Developers rely on software agents for decisions as well as code changes. TaskSift turns historical developer-agent sessions into executable cases that preserve the developer's intent, the timing of user information, and evidence-grounded acceptance criteria. From 12,660 internal product and open source sessions, it builds SiftBench, with 6,952 cases in ten categories. Analytical tasks make up two thirds of the internal product cases. Four agent configurations complete 56.0% of internal product Analytical runs, about 17 pp below the open source rate, and staged information delivery lowers completion.

Looking ahead, we plan to refresh SiftBench continuously from new sessions so that it tracks how developers use agents over time, and to apply TaskSift to sessions from other organizations. The Ground Truth Cards also record which evidence an incomplete answer missed. We plan to use this

signal to study how agents gather evidence and to help them ask for the information a decision requires.

REFERENCES

[1] H. Hao *et al.*, "An empirical study on developers' shared conversations with ChatGPT in GitHub pull requests and issues," *Empirical Software Engineering*, vol. 29, no. 6, 2024, art. no. 150. [Online]. Available: https://doi.org/10.1007/s10664-024-10540-x

[2] R. Ehsani, S. Pathak, E. Parra, S. Haiduc, and P. Chatterjee, "What characteristics make ChatGPT effective for software issue resolution? an empirical study of task, project, and conversational signals in GitHub issues," *Empirical Software Engineering*, vol. 31, no. 1, 2026, art. no. 22. [Online]. Available: https://doi.org/10.1007/s10664-025-10745-8

[3] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-world Github Issues?" in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[4] X. Deng *et al.*, "SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?" in *International Conference on Machine Learning*, ser. PMLR, vol. 306, 2026, pp. 23 917–23 934. [Online]. Available: https://proceedings.mlr.press/v306/deng26f.html

[5] W. Peng, Y. Shi, Y. Wang, X. Zhang, B. Shen, and X. Gu, "SWE-QA: Can Language Models Answer Repository-level Code Questions?" in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 8230–8245. [Online]. Available: https://aclanthology.org/2026.findings-acl.402/

[6] Y. Yang *et al.*, "RepoProbe: Benchmarking Architecture-Aware Repository Comprehension with Checklists," arXiv:2608.04783, 2026. [Online]. Available: https://arxiv.org/abs/2608.04783

[7] M. Zhou and A. Carderera de Diego, "ReviewBench: An open benchmark for AI code review," GitHub Blog, 2026, oct. 5, 2026. [Online]. Available: https://github.blog/ai-and-ml/reviewbench-an-open-benchmark-for-ai-code-review/

[8] A. Gong *et al.*, "ORCA-bench: How Ready Are Language Model Agents for Oncall?" arXiv:2607.28545, 2026. [Online]. Available: https://arxiv.org/abs/2607.28545

[9] Z. Lv *et al.*, "RealClawBench: Live OpenClaw Benchmarks from Real Developer-Agent Sessions," arXiv:2606.03889, 2026. [Online]. Available: https://arxiv.org/abs/2606.03889

[10] Y. Wu *et al.*, "SWE-Together: Evaluating Coding Agents in Interactive User Sessions," arXiv:2606.29957, 2026. [Online]. Available: https://arxiv.org/abs/2606.29957

[11] M. Kim, S. Garg, B. Ray, V. Kumar, and A. Deoras, "CodeAssistBench (CAB): Dataset & Benchmarking for Multi-turn Chat-Based Code Assistance," in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: https://openreview.net/forum?id=2R6y4Ku9kG

[12] J. Baumann, V. Padmakumar, X. Li, J. Yang, D. Yang, and S. Koyejo, "SWE-chat: Coding Agent Interactions From Real Users in the Wild," in *Conference on Language Modeling*, 2026. [Online]. Available: https://arxiv.org/abs/2604.20779

[13] DataClaw Community, "DataClaw: Publish Coding Agent Chat History as Hugging Face Datasets," GitHub repository, 2026. [Online]. Available: https://github.com/peteromallet/dataclaw

[14] M. Zechner, "pi-share-hf: Collect, Review, and Upload Redacted Pi Session Files to a Hugging Face Dataset," GitHub repository, 2026. [Online]. Available: https://github.com/badlogic/pi-share-hf

[15] J. Yang *et al.*, "SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?" in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=riTiq3i21b

[16] ——, "SWE-smith: Scaling Data for Software Engineering Agents," in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: https://papers.nips.cc/paper_files/paper/2025/hash/8b86cf5ace600c48fd188efbb8dedec8-Abstract-Datasets_and_Benchmarks_Track.html

[17] S. Miserendino, M. Wang, T. Patwardhan, and J. Heidecke, "SWE-Lancer: Can Frontier LLMs Earn $1 Million from Real-World Freelance Software Engineering?" in *International Conference on Machine Learning*, 2025. [Online]. Available: https://openreview.net/forum?id=xZXhFg43EI

[18] S. Cai *et al.*, "SWE-QA-Pro: A Representative Benchmark and Scalable Training Recipe for Repository-Level Code Understanding," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 16 958–16 999. [Online]. Available: https://aclanthology.org/2026.findings-acl.837/

[19] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 21 558–21 572. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract.html

[20] K. Li *et al.*, "RepoLaunch: Automating Build and Management of Code Repositories across Languages and Platforms," arXiv:2603.05026, 2026. [Online]. Available: https://arxiv.org/abs/2603.05026

[21] H. Qi *et al.*, "Change2Task: From Repository Changes to Executable Coding Agent Tasks and Environments," arXiv:2607.28591, 2026. [Online]. Available: https://arxiv.org/abs/2607.28591

[22] T. Gebru *et al.*, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021. [Online]. Available: https://doi.org/10.1145/3458723

[23] P. Liang *et al.*, "Holistic Evaluation of Language Models," *Transactions on Machine Learning Research*, 2023. [Online]. Available: https://arxiv.org/abs/2211.09110

[24] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[25] J. Austin *et al.*, "Program Synthesis with Large Language Models," arXiv:2108.07732, 2021. [Online]. Available: https://arxiv.org/abs/2108.07732

[26] N. Jain *et al.*, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=chfJJYC3iL

[27] T. Y. Zhuo *et al.*, "BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=YrycTjllL0

[28] S. Lu *et al.*, "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation," in *Advances in Neural Information Processing Systems*, 2021. [Online]. Available: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round1.html

[29] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=pPjZIOuQuF

[30] Y. Ding *et al.*, "CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion," in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://openreview.net/forum?id=wgDcbBMSfh

[31] H. Qi, X. Xu, Y. Huang, J. Shang, and C. Wang, "WitnessGym: Benchmarking Coding Agents on the Construction of Bug Witnesses," arXiv:2609.36635, 2026. [Online]. Available: https://arxiv.org/abs/2609.36635

[32] S. Sun, "SQBench: A Benchmark for Evaluating Task Delivery by Language-Model Agents in Production-Oriented Workflows," arXiv:2607.23123, 2026. [Online]. Available: https://arxiv.org/abs/2607.23123

[33] L. Li *et al.*, "InfiBench: Evaluating the Question-Answering Capabilities of Code Large Language Models," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://proceedings.nips.cc/paper_files/paper/2024/hash/e888eb9400fe14bb70e057aa1d719188-Abstract-Datasets_and_Benchmarks_Track.html

[34] Cursor Research, "Composer 2 Technical Report," arXiv:2603.24477, 2026. [Online]. Available: https://arxiv.org/abs/2603.24477

[35] Tencent WorkBuddy Bench Team *et al.*, "Tencent WorkBuddy Bench: A Multi-Domain Coding-Agent Benchmark with Contamination-Resistant Task Construction," arXiv:2607.20911, 2026. [Online]. Available: https://arxiv.org/abs/2607.20911

[36] J.-Y. Kim and T.-Y. Lee, "BTS-AgentBench: A Deterministic, Replayable Pipeline from Read-Only Telemetry Logs to Agent Benchmarks," arXiv:2608.27334, 2026. [Online]. Available: https://arxiv.org/abs/2608.27334

[37] P. Laban, H. Hayashi, Y. Zhou, and J. Neville, "LLMs Get Lost In Multi-Turn Conversation," in *International Conference on Learning Representations*, 2026. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/hash/59f6421e64707225fdf5b28840679a07-Abstract-Conference.html

[38] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html

[39] V. Barres, H. Dong, S. Ray, X. Si, and K. R. Narasimhan, "τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment," in *International Conference on Machine Learning*, ser. PMLR, vol. 306, 2026, pp. 6736–6782. [Online]. Available: https://proceedings.mlr.press/v306/barres26a.html

[40] J. Lu *et al.*, "ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 1160–1183. [Online]. Available: https://aclanthology.org/2025.findings-naacl.65/

[41] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[42] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 46 534–46 594. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[43] J. Huang *et al.*, "Large Language Models Cannot Self-Correct Reasoning Yet," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.01798

[44] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[45] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=mXpq6ut8J3

[46] X. Wang *et al.*, "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://arxiv.org/abs/2407.16741

[47] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-Based Software Engineering Agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025. [Online]. Available: https://doi.org/10.1145/3715754

[48] X. Liu *et al.*, "AgentBench: Evaluating LLMs as Agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2308.03688

[49] C. Ma *et al.*, "AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://arxiv.org/abs/2401.13178

[50] S. Zhou *et al.*, "WebArena: A Realistic Web Environment for Building Autonomous Agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=oKn9c6ytLx

[51] A. Drouin *et al.*, "WorkArena: How Capable are Web Agents at Solving Common Knowledge Work Tasks?" in *International Conference on Machine Learning*, 2024, pp. 11 642–11 662. [Online]. Available: https://proceedings.mlr.press/v235/drouin24a.html

[52] T. Xie *et al.*, "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 52 040–52 094. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html

[53] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG Evaluation using Gpt-4 with Better Human Alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2511–2522. [Online]. Available: https://aclanthology.org/2023.emnlp-main.153/

[54] S. Kim *et al.*, "Prometheus: Inducing Fine-grained Evaluation Capability in Language Models," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.08491

[55] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[56] S. Tan *et al.*, "JudgeBench: A Benchmark for Evaluating LLM-based Judges," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://arxiv.org/abs/2410.12784

[57] Z. Zeng, J. Yu, T. Gao, Y. Meng, T. Goyal, and D. Chen, "Evaluating Large Language Models at Evaluating Instruction Following," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.07641

[58] S. Min *et al.*, "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 12 076–12 100. [Online]. Available: https://aclanthology.org/2023.emnlp-main.741/

[59] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling Large Language Models to Generate Text with Citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 6465–6488. [Online]. Available: https://aclanthology.org/2023.emnlp-main.398/

[60] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 9004–9017. [Online]. Available: https://aclanthology.org/2023.emnlp-main.557/

[61] M. I. Gorinova, M. Baker, A. Heineike, M. Shaposhnikov, R. Willoughby, and D. Knox, "Position: Coding Benchmarks Are Misaligned with Agentic Software Engineering," arXiv:2606.17799, 2026. [Online]. Available: https://arxiv.org/abs/2606.17799